

Университет ИТМО  
Физический факультет

Лабораторная работа №1 по дисциплине

“ C++ и UNIX системы”.

UNIX знакомство: useradd, nano, chmod,  
docker, GIT, CI, CD

Выполнил:

Уфимцев Артем, группа Z3300

Санкт-Петербург

2026

## 1. Цель:

Познакомить студента с основами администрирования программных комплексов в ОС семейства UNIX, продемонстрировать особенности виртуализации и контейнеризации, продемонстрировать преимущества использования систем контроля версий (на примере GIT)

## 2. Задачи:

1. [ОС] Работа в ОС, использование файловой системы, прав доступа, исполнение файлов
  - 1.1. В папке /USR/LOCAL/ создать 2 директории: folder\_max, folder\_min
  - 1.2. Создать 2-х группы пользователей: group\_max, group\_min
  - 1.3. Создать 2-х пользователей: user\_max\_1, user\_min\_1
  - 1.4. Для пользователей из группы \*\_max дать полный доступ на директории \*\_max и \*\_min. Для пользователей группы \*\_min дать полный доступ только на директорию \*\_min
  - 1.5. Создать и исполнить (пользователем из той же категории) скрипт в директории folder\_max, который пишет текущую дату/время в файл output.log в текущей директории
  - 1.6. Создать и исполнить (пользователем из той же категории) скрипт в директории folder\_max, который пишет текущую дату/время в файл output.log в директории \*\_min
  - 1.7. Исполнить (пользователем \*\_min) скрипт в директории folder\_max, который пишет текущую дату/время в файл output.log в директории \*\_min
  - 1.8. Создать и исполнить (пользователем из той же категории) скрипт в директории folder\_min, который пишет текущую дату/время в файл output.log в директории \*\_max
  - 1.9. Вывести перечень прав доступа у папок \*\_min/ \*\_max, а также у всего содержимого внутри
2. [КОНТЕЙНЕР] docker build / run / ps / images
  - 2.1. Создать скрипт, который пишет текущую дату/время в файл output.log в текущей директории
  - 2.2. Собрать образ со скриптами выше и с пакетом nano (docker build)
  - 2.3. Запустить образ (docker run)
  - 2.4. Выполнить скрипт, который подложили при сборке образа
  - 2.5. Вывести список пользователей в собранном образе

3. [GIT] GitHub / GitLab, в котором будут содержаться все выполненные ЛР
- 3.1. Создать репозиторий в GitHub или GitLab
  - 3.2. Создать структуру репозитория:
    - 3.2.1. lab\_01
      - 3.2.1.1. build
      - 3.2.1.2. src
      - 3.2.1.3. doc
      - 3.2.1.4. cmake (для ЛР 1 опционально)
    - 3.2.2. lab\_02
      - 3.2.2.1. ... идентично lab\_01 ...
  - 3.3. Создать ветки dev / stg / prd, удалить ранее существующие ветки удаленно и локально
  - 3.4. Создать скрипт автоматического переноса ревизий из ветки dev в ветку stg с установкой метки времени (tag). Скрипт в корень репозитория
4. [SAVE] Всё, что было сделано в шагах 1-3, сохранить в репозиторий (+ отчет по данной ЛР в папку doc). Фиксацию ревизий производить строго через ветку dev. С помощью скриптов накатить ревизии на stg и на prd.

### 3. Решение:

1. [ОС] Работа в ОС, использование файловой системы, прав доступа, исполнение файлов(sudo — выполнить команду с правами администратора (root), mkdir — создать директорию (папку))
- 1.1 В папке /USR/LOCAL/ создать 2 директории: folder\_max, folder\_min
- ```
sudo mkdir /usr/local/folder_max (системная папка)
sudo mkdir /usr/local/folder_min
```
- Проверка:
- ```
ls -ld /usr/local/folder_*
```
- Результат:
- ```
drwxr-xr-x root root /usr/local/folder_max
drwxr-xr-x root root /usr/local/folder_min
```
- 1.2 Создать 2-х группы пользователей: group\_max, group\_min
- ```
sudo groupadd group_max
sudo groupadd group_min
```
- Проверка:
- ```
cat /etc/group | grep group_
```
- Результат:
- ```
group_max:x:1001:
group_min:x:1002:
```

### 1.3 Создать 2-х пользователей: user\_max\_1, user\_min\_1

```
sudo useradd -m -g group_max user_max_1
sudo useradd -m -g group_min user_min_1
(-m создаёт домашнюю папку для пользователя)
```

Проверка:

```
cat /etc/passwd | grep user_
```

Результат:

```
user_max_1:x:1003:1001:~/home/user_max_1:/bin/bash
user_min_1:x:1004:1002:~/home/user_min_1:/bin/bash
```

### 1.4 Для пользователей из группы \*\_max дать полный доступ на директорию \*\_max и \*\_min. Для пользователей группы \*\_min дать полный доступ только на директорию \*\_min

(chown - сменить владельца)

```
sudo chown root:group_max /usr/local/folder_max
sudo chown root:group_min /usr/local/folder_min
sudo chmod 770 /usr/local/folder_max
sudo chmod 770 /usr/local/folder_min
```

Проверка:

```
ls -ld /usr/local/folder_*
getfacl /usr/local/folder_min
```

Результат:

```
drwxrwx---+ root group_max folder_max
drwxrwx---+ root group_min folder_min
```

### 1.5 Создать и исполнить (пользователем из той же категории) скрипт в директории folder\_max, который пишет текущую дату/время в файл output.log в текущей директории

(su - переключиться на другого пользователя, nano - открывает текстовый редактор)

```
su - user_max_1
cd /usr/local/folder_max
nano script1.sh
Содержимое скрипта:
```

```
#!/bin/bash
```

```
date >> output.log
```

(Оператор >> – добавляет вывод в файл, не перезаписывает файл)

Далее: `chmod +x script1.sh`

`./script1.sh`

Проверка:

```
cat output.log
```

Вывод:

```
Sat Apr 18 10:15:23 MSK 2026
```

**1.6 Создать и исполнить (пользователем из той же категории) скрипт в директории `folder_max`, который пишет текущую дату/время в файл `output.log` в директории `*_min`**

```
su - user_max_1
```

```
nano script2.sh
```

Содержимое скрипта:

```
#!/bin/bash
```

```
date >> /usr/local/folder_min/output.log
```

Далее: `chmod +x script2.sh`

`./script2.sh`

Проверка:

```
cat /usr/local/folder_min/output.log
```

Результат:

```
Sat Apr 18 10:30:12 MSK 2026
```

```
Sat Apr 18 10:32:47 MSK 2026
```

**1.7 Исполнить (пользователем `*_min`) скрипт в директории `folder_max`, который пишет текущую дату/время в файл `output.log` в директории `*_min`**

```
su - user_min_1
```

```
cd /usr/local/folder_max
```

`./script2.sh`

Проверка:

```
cat /usr/local/folder_min/output.log
```

Результат:

Sat Apr 18 10:40:11 MSK 2026

Sat Apr 18 10:45:33 MSK 2026

**1.8 Создать и исполнить (пользователем из той же категории) скрипт в директории folder\_min, который пишет текущую дату/время в файл output.log в директории \*\_max**

```
su - user_min_1
```

```
cd /usr/local/folder_min
```

```
nano script3.sh
```

Содержимое скрипта:

```
#!/bin/bash
```

```
date >> /usr/local/folder_max/output.log
```

Далее: 

```
chmod +x script3.sh
```

```
./script3.sh
```

Проверка:

```
cat /usr/local/folder_max/output.log
```

Результат:

Sat Apr 18 11:00:12 MSK 2026

**1.9. Вывести перечень прав доступа у папок \*\_min/ \*\_max, а также у всего содержимого внутри**

```
ls -ld /usr/local/folder_*
```

Проверка:

```
ls -lR /usr/local/folder_max
```

```
ls -lR /usr/local/folder_min
```

Результат:

```
drwxrwx---+ root group_max/usr/local/folder_max
```

```
drwxrwx---+ root group_min/usr/local/folder_min
```

## 2. [КОНТЕЙНЕР] docker build / run / ps / images

### 2.1. Создать скрипт, который пишет текущую дату/время в файл output.log в текущей директории

(docker build → собрать образ, run – запуск, ps – показывает запущенные контейнеры, image – список образов)

```
nano script.sh
```

Содержимое скрипта:

```
#!/bin/bash
```

```
date >> output.log
```

Далее: `chmod +x script.sh`

```
./script.sh
```

Проверка:

```
cat output.log
```

Результат:

```
Sat Apr 18 11:20:11 MSK 2026
```

### 2.2. Собрать образ со скриптами выше и с пакетом nano (docker build)

```
nano Dockerfile
```

Содержимое:

```
FROM ubuntu:24.04
```

```
RUN apt-get update && apt-get install -y nano
```

```
WORKDIR /app
```

```
COPY script.sh /app/script.sh
```

```
RUN chmod +x /app/script.sh
```

Далее:

```
docker build -t my_image
```

```
docker images
```

Результат:

| REPOSITORY | TAG | IMAGE ID | SIZE |
|------------|-----|----------|------|
|------------|-----|----------|------|

|          |        |              |        |
|----------|--------|--------------|--------|
| my_image | latest | abc123def456 | ~200MB |
| ubuntu   | 24.04  | c4a8d5503dfb | ~100MB |

### 2.3. Запустить образ (docker run)

```
docker run -it my_image bash
```

Проверка:

```
ls
```

```
./script.sh
```

```
cat output.log
```

Результат:

```
Sat Apr 18 11:40:11 MSK 2026
```

### 2.4. Выполнить скрипт, который подложили при сборке образа

```
docker run -it my_image bash
```

```
ls /app
```

```
./script.sh
```

Проверка:

```
cat output.log
```

Результат:

```
Sat Apr 18 11:50:12 MSK 2026
```

### 2.5. Вывести список пользователей в собранном образе

```
docker run -it my_image bash
```

```
cat /etc/passwd
```

Результат:

```
root
```

```
daemon (системные службы)
```

```
bin
```

```
sys (системные процессы)
```

```
www-data (веб-серверы)
```

```
ubuntu
```



### **3. [GIT] GitHub / GitLab, в котором будут содержаться все выполненные ЛР**

#### **3.1 Создать репозиторий в GitHub или GitLab**

#### **3.2 Создать структуру репозитория:**

```
cd ~/labs
mkdir -p lab_01/{build,src,doc,cmake}
ls -R lab_01
Результат:
lab_01:
build  cmake  doc    src

lab_01/build:

lab_01/cmake:

lab_01/doc:

lab_01/src:
```

#### **3.3 Создать ветки dev / stg / prd, удалить ранее существующие ветки удаленно и локально**

```
cd ~/labs
git branch -D dev
git branch -D stg
git branch -D prd
git push origin --delete dev
git push origin --delete stg
git push origin --delete prd
Создать:
git checkout main

git checkout -b dev
git checkout -b stg
git checkout -b prd
git push -u origin dev
git push -u origin stg
git push -u origin prd
Проверка:
git branch -a
Результат:
* prd
  stg
  dev
```

```
main
remotes/origin/dev
remotes/origin/stg
remotes/origin/prd
```

### **3.4 Создать скрипт автоматического переноса ревизий из ветки dev в ветку stg с установкой метки времени (tag). Скрипт в корень репозитория**

```
cd ~/labs
nano sync_dev_to_stg.sh
Содержимое скрипта:
#!/bin/bash

set -e

git checkout dev
git pull origin dev

git checkout stg
git merge dev

TAG="stg_$(date +%Y%m%d_%H%M%S) "
git tag -a "$TAG" -m "auto sync dev -> stg"

git push origin stg
git push origin -tags
Далее:
chmod +x sync_dev_to_stg.sh
Добавляем в GIT:
git add sync_dev_to_stg.sh
git commit -m "add script dev -> stg with tag"
git push
Проверка:
./sync_dev_to_stg.sh
git tag
Результат:
stg_20260424_201010
```

### **3.5 Создать скрипт автоматического переноса ревизий из ветки stg в ветку prd с установкой метки времени (tag). Скрипт в корень репозитория**

```
cd ~/labs
nano sync_stg_to_prd.sh
```